
Introduction to Neural Networks

ECON 8310: Business Forecasting — Lecture 7

Department of Economics

University of Nebraska at Omaha

August 25, 2026

Lecture Outline

- 1 From Trees to Neural Networks
- 2 Feedforward Networks
- 3 Training: Loss and Backpropagation
- 4 PyTorch Implementation
- 5 Key Takeaways

From Trees to Neural Networks

Trees predict in steps. Networks predict in curves — and that difference buys less than you would expect on tabular data, and everything on sequences.

Why Go Beyond Trees?

A tree-based model treats every row as independent. Order is not part of the model — everything it knows about time, you put there by hand, as lag columns and rolling means. And its prediction is **piecewise constant**, which is why in Lab 4 the tree could not predict above the largest value it had seen.

A neural network learns a **smooth, continuous** function instead. Its architecture is not fixed: the same machinery adapts to sequences, images and text, and it can learn its own intermediate representations rather than being handed engineered ones.

This is not an upgrade. On the tabular problems of Homework 3 and 4, gradient boosting and a tuned LASSO were hard to beat, and a network will often lose to both. Reach for one when the data is a *sequence*, when the raw input is an image or audio, or when n is very large. Otherwise you are paying in tuning time for accuracy you will not get.

So today is about the machinery. Lectures 8 and 9 are where it earns its place.

From Linear Regression to a Single Neuron

Start from something you already know. Linear regression predicts $\hat{y} = \mathbf{w}^\top \mathbf{x} + b$.

A **neuron** is that same expression with one addition — a nonlinear **activation** $\sigma(\cdot)$ wrapped around it:

$$a = \sigma(\mathbf{w}^\top \mathbf{x} + b)$$

Activation	Formula	Where it is used
ReLU	$\max(0, z)$	Hidden layers — the default
Sigmoid	$1/(1 + e^{-z})$	Binary outputs, and LSTM gates (Lecture 9)
Tanh	$\tanh(z)$	Sequence states
Linear	z	The output layer, for regression

ReLU makes the intuition literal: the neuron asks *“is this pattern present?”* If the evidence is positive it passes a signal through; if not it outputs exactly zero and stays silent.

Why the Nonlinearity Is the Whole Trick

It is worth being precise about what $\sigma(\cdot)$ buys, because it is easy to assume that depth alone is what makes a network powerful. It is not.

Stacking linear layers gains you nothing

A linear function of a linear function is linear. Compose two layers with no activation between them and you get $\mathbf{W}_2(\mathbf{W}_1\mathbf{x}) = (\mathbf{W}_2\mathbf{W}_1)\mathbf{x}$ — a single matrix, and therefore a single linear model with extra steps.

Insert a nonlinearity between the layers and the picture changes completely. The **universal approximation theorem** says a network with one hidden layer can approximate *any* continuous function to arbitrary accuracy, given enough neurons.

Read that claim carefully. It says a network *can* represent the function — not that your optimizer will find it, that you have the data to identify it, or that it will generalize. Existence is not estimation, the same gap as between the optimal tree and the greedy one CART builds.

Feedforward Networks

Stack layers of neurons, and each layer learns a higher-level representation of the one below it.

Feedforward Network: Layers and Forward Pass

A **feedforward network** — an FFN, or multilayer perceptron — is three things in sequence. An **input layer** of raw features $\mathbf{x} \in \mathbb{R}^p$; some number of **hidden layers**, each applying $\mathbf{a}^{(\ell)} = \sigma(\mathbf{W}^{(\ell)}\mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)})$; and an **output layer** that is linear for regression.

Two words you need: **width** is the number of neurons in a layer, **depth** is the number of hidden layers. Together they set the model's capacity.

Sensible starting sizes for forecasting: 2–3 hidden layers, 64–256 neurons each, ReLU activations, dropout between 0.2 and 0.5. Start at the small end of every range.

Bigger is not better on its own. A wider or deeper network needs more data and more regularization to reach the same generalization, so add capacity only when validation error says the current model is genuinely too simple — not because it is available.

How Big Is This Model, Actually?

Capacity is easy to add by accident, so it is worth counting once. A layer mapping m inputs to n outputs holds $m \times n$ weights plus n biases.

Take a network for our weekly panel: 46 engineered features in, two hidden layers of 128, one output.

Layer	Shape	Parameters
Input $\rightarrow$ hidden 1	46×128	6,016
Hidden 1 $\rightarrow$ hidden 2	128×128	16,512
Hidden 2 $\rightarrow$ output	128×1	129
Total		22,657

Twenty-two thousand parameters on five thousand rows — four per observation.

Lecture 6 called sixty predictors on a hundred and fifty observations a crisis. This is far past that ratio, and networks train here routinely. What makes it survivable is not the architecture but the regularization, next.

Training: Loss and Backpropagation

Compute the error, propagate it backward, adjust every weight by its share of the blame.
Repeat.

Training: Loss, Gradients, and the Update Step

Training is one loop, repeated. **First**, compute the loss — for regression, mean squared error, $\mathcal{L} = \frac{1}{n} \sum_i (y_i - \hat{y}_i)^2$.

Second, backpropagation: work backward through the network, using the chain rule to assign each weight its share of the error. **Third**, nudge every weight against its gradient:

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{W}}, \quad \eta = \text{learning rate}$$

Stochastic gradient descent

Rather than using the whole dataset for each update, use a random **mini-batch** of 32–256 rows. Noisier steps, far more of them, and much faster. One pass through every batch is an **epoch**.

You never differentiate by hand. `loss.backward()` does the backward pass, and `torch.optim.Adam(lr=1e-3)` beats plain SGD — it adapts the step size per parameter.

Preventing Overfitting — Where Lecture 6 Returns

Twenty-two thousand parameters on five thousand rows will memorize the training set unless something stops it. Three things do, and **you have met two of them already**.

<code>weight_decay</code>	This is Ridge. An ℓ_2 penalty on the weights, under a different name. <code>Adam(..., weight_decay=1e-4)</code>
<code>nn.Dropout(p)</code>	Randomly zero a fraction p of neuron outputs on each forward pass, so no neuron can depend on a specific other one being present. All neurons are active at test time.
Early stopping	Watch validation loss each epoch; stop when it has not improved for k epochs, and keep the weights from the <i>best</i> epoch, not the last.

That is the fifth appearance of one idea. The GAM smoothing penalty in Lecture 3, cost-complexity pruning in Lecture 4, `reg_lambda` on leaf weights in Lecture 5, Ridge and LASSO in Lecture 6 — and now weight decay. Fit against complexity, one parameter setting the exchange rate, chosen by validation.

PyTorch Implementation

Four pieces: a Dataset, a DataLoader, a model, and a training loop.

Wrapping your arrays

```
from torch.utils.data import Dataset, DataLoader

class ForecastDataset(Dataset):

    def __init__(self, X, y):

        self.X = torch.tensor(X, dtype=torch.float32)

        self.y = torch.tensor(y, dtype=torch.float32)

    def __len__(self): return len(self.X)

    def __getitem__(self, i): return self.X[i], self.y[i]

loader = DataLoader(ForecastDataset(X_tr, y_tr),

                    batch_size=64, shuffle=False)
```

Dataset fetches *one* example; DataLoader handles batching.

Note shuffle=False. Nearly every tutorial sets it True — right for images, wrong here. It destroys the ordering your lag features depend on.

PyTorch: Defining the Model

A two-layer network with dropout

```
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(n_features, 128), nn.ReLU(), nn.Dropout(0.3),
    nn.Linear(128, 128),      nn.ReLU(), nn.Dropout(0.3),
    nn.Linear(128, 1))
```

Read it top to bottom and it is the diagram: a linear map, a nonlinearity, some dropout, again, then a linear output with no activation because this is regression.

`nn.Sequential` chains layers in order and is enough for anything in this lecture. When an architecture needs branching or a skip connection — Lecture 8 — you subclass `nn.Module` and write `forward()` yourself.

`nn.Linear(in, out)` holds the weight matrix and bias for one layer. That is where the 22,657 parameters live.

PyTorch: The Training Loop

The four steps, per mini-batch

```
opt = torch.optim.Adam(model.parameters(), lr=1e-3, weight_decay=1e-4)
lossfn = nn.MSELoss()

for epoch in range(n_epochs):
    model.train()

    for xb, yb in loader:
        opt.zero_grad()           # 1 clear old gradients

        loss = lossfn(model(xb).squeeze(), yb) # 2 forward

        loss.backward()           # 3 backward

        opt.step()                # 4 update weights
```

Every PyTorch training loop is those four lines with more around them.

The one that bites: forget `zero_grad()` and PyTorch *accumulates* gradients rather than replacing them. No error, no warning — just a model that trains badly.

Key Takeaways

Where a network belongs in the toolkit you now have.

Neural Networks vs. Tree Methods (James et al. 2023)

Method	Sequences	Extrapolates	Tuning	Best for
LASSO / Ridge	no	yes	light	Many correlated predictors
Random forest	no	no	light	Tabular, robust default
XGBoost	no	no	moderate	Tabular, best-in-class
FFN	no	yes	heavy	Smooth functions, large n
RNN / LSTM	yes	yes	heavy	Sequences (Lecture 9)

An FFN on lag features is doing the same job as XGBoost on lag features — same inputs, same flat table, no notion of order. It is not obviously the better tool, and on the evidence of Homework 4 it usually is not.

What changes in the next two lectures is the *architecture*: convolutions that see local structure, and recurrence that carries state through time. That is when the sequence column above starts saying yes, and it is the reason to learn this machinery now.

Lecture 7: Key Takeaways

1. A **neuron** is linear regression wrapped in a nonlinear activation. Without the nonlinearity, stacking layers collapses back to one linear model.
2. An **FFN** stacks those layers. Width and depth set capacity, and capacity accumulates faster than you expect — 22,657 parameters on 5,000 rows.
3. **Training** is one loop: `loss` → `backward()` → `step()`, on mini-batches, using Adam. You never differentiate by hand.
4. **Regularization is not new.** Weight decay is Ridge; dropout and early stopping do the same job by other means. Fifth appearance of one idea.
5. For time series: **do not shuffle** the DataLoader, split by time, and call `zero_grad()` every batch.
6. An FFN on engineered features is a competitor to XGBoost, **not an upgrade**. Architecture is what earns its keep — next two lectures.

Next: CNN architectures — convolution, pooling, and 1D convolutions for time series.

References I



James, Gareth et al. (2023). *An Introduction to Statistical Learning with Applications in Python*. 1st. New York: Springer. URL: <https://www.statlearning.com/>.